LAPORAN PRAKTIKUM PEMROGRAMAN BERBASIS OBJEK

Minggu Ke-5

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Nurfiah S.ST, M.Kom

Asisten Praktikum: Siti Fadhilah Rahmi



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI ..... **i**

KATA PENGANTAR ..... **ii**

BAB I PENDAHULUAN **1**

1.1 Latar Belakang **1**

1.2 Tujuan Praktikum **1**

1.3 Manfaat Praktikum **1**

BAB II PEMBAHASAN **2**

2.1 Refaktor Method tarikTunai pada Class Main **2**

2.2 Melakukan Overriding pada RekeningGiro **2**

BAB III TUGAS **4**

3.1 Fitur Transfer Antar Rekening **4**

3.2 Integrasi ke CLI Utama **4**

BAB IV KESIMPULAN **6**

4.1 Ringkasan Hasil Praktikum **6**

4.2 Link Github **6**

DAFTAR PUSTAKA ..... **iii**

KATA PENGANTAR

Puji syukur kehadirat Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Pemrograman Berbasis Objek.

Ucapan terima kasih disampaikan kepada Bapak Nurfiah S.ST, M.Kom. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam Pemrograman Berorientasi Objek (PBO), polimorfisme memungkinkan satu antarmuka menggunakan nama method yang sama untuk dieksekusi dengan cara yang berbeda-beda tergantung pada tipe objek aslinya pada saat runtime atau waktu eksekusi. Melalui Method Overriding, kelas anak dapat mendefinisikan ulang method milik kelas induk, dan juga melalui mekanisme Dynamic Method Dispatch, sistem Java dapat secara otomatis menentukan logika mana yang harus dipanggil

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

- Memahami konsep Polimorfisme dan Dynamic Method Dispatch
- Melakukan Method Overriding untuk memodifikasi behavior kelas induk pada kelas anak
- Merancang method yang dapat menerima argumen polimorfik yaitu objek dari berbagai subclass

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

- Meningkatkan pemahaman dalam merancang sistem yang fleksibel dengan mengandalkan polimorfisme dan Dynamic Method Dispatch
- Memiliki kemampuan untuk mengimplementasikan Method Overriding sesuai kebutuhan logika bisnis spesifik suatu subclass
- Mampu menciptakan interaksi antar-objek secara aman melalui manipulasi parameter polimorfik pada aplikasi perbankan

BAB 2

PEMBAHASAN

2.1 Refaktor Method tarikTunai pada Class Main

```
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
            return false;
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi.
Saldo Anda: Rp" + saldo);
            return false;
        } else {
            saldo -= nominal;

            String idTrx = "TRX-T-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

            System.out.println("Tarik tunai Rp" + nominal + " berhasil.
Sisa saldo: Rp" + saldo);
            return true;
        }
    }
```

Langkah awal yang dilakukan pada praktikum ini adalah merombak method tarikTunai pada kelas induk Rekening. Tipe kembalian (return type) yang semula berupa void dimodifikasi menjadi boolean agar fungsionalitas penarikan dapat mengembalikan nilai true jika berhasil dan false jika gagal

2.2 Melakukan Overriding pada RekeningGiro

```
    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik Giro harus lebih dari
0!");
            return false;
        }
        // Logika Khusus Giro: Boleh menarik asalkan nominal tidak melebihi
(saldo + batasOverdraft)
        else if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Transaksi Gagal: Melebihi batas overdraft!
Maksimal penarikan: Rp" + (saldo + batasOverdraft));
            return false;
        }
        else {
```

```
            saldo -= nominal; // Saldo bisa bernilai negatif di sini
(meminjam overdraft)

            String idTrx = "TRX-G-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro",
nominal));

            System.out.println("Tarik tunai Giro Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
            return true;
        }
    }
```

Method Overriding diterapkan di dalam kelas RekeningGiro dengan menimpa fungsi tarikTunai bawaan dari kelas induknya. @Override digunakan untuk memastikan kepada compiler bahwa metode ini sengaja ditimpa. Logika di dalamnya dimodifikasi khusus untuk entitas Giro, di mana nasabah diizinkan melakukan penarikan hingga saldo bernilai negatif, selama tidak melewati batas overdraft yang telah ditetapkan

BAB 3

TUGAS

3.1 Fitur Transfer Antar Rekening

```
    public void transfer(Rekening tujuan, double nominal, String pinInput)
{
        if (this.otentikasi(pinInput) == false) {
            System.out.println("Transfer Gagal: PIN yang Anda masukkan
salah!");
            return;
        }

        boolean suksesTarik = this.tarikTunai(nominal);

        if (suksesTarik == true) {
            tujuan.setorTunai(nominal);
            System.out.println("Transfer sebesar Rp" + nominal + " ke " +
tujuan.getNamaPemilik() + " berhasil diproses.");
        }
    }
```

Method transfer() ditambahkan ke dalam class Rekening. Perbedaan method ini terletak pada penggunaannya menerima parameter polimorfik berupa kelas induk Rekening tujuan, yang memungkinkannya untuk memproses transfer baik ke objek Tabungan maupun Giro tanpa perlu membedakan tipe spesifiknya secara manual. Pemanggilan this.tarikTunai(nominal) menggunakan fitur Dynamic Method Dispatch, di mana sistem akan menyesuaikan aturan penarikan (apakah menggunakan logika tabungan biasa atau logika hutang pada giro) berdasarkan wujud asli dari entitas akunAktif

3.2 Integrasi ke CLI Utama

```
                case 9:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum membuka
rekening!");
                    } else {
                        System.out.print("Masukkan Nomor Rekening Tujuan:
");
                        String rekTujuan = input.nextLine();

                        Rekening akunTujuan = null;

                        for (Rekening rek : daftarRekening) {
```

```
                            if (rek.getNomorRekening().equals(rekTujuan)) {
                                akunTujuan = rek;
                                break;
                            }
                        }

                        if (akunTujuan == null) {
                            System.out.println("Transfer Gagal: Rekening
tujuan tidak ditemukan!");
                        } else {
                            System.out.print("Masukkan nominal transfer:
Rp");
                            double nominalTf = input.nextDouble();
                            input.nextLine();

                            System.out.print("Masukkan PIN Anda untuk
konfirmasi: ");
                            String pinTf = input.nextLine();

                            akunAktif.transfer(akunTujuan, nominalTf,
pinTf);
                        }
                    }
                    break;
```

Implementasi integrasi metode transfer ke CLI di file Main.java. Program akan memvalidasi input nomor rekening target dengan melakukan iterasi menggunakan struktur Enhanced For-Loop pada daftarRekening. Jika rekening tujuan berhasil ditemukan, alur program akan diteruskan untuk menanyakan nominal dan verifikasi PIN sebelum akhirnya memanggil fungsi transfer

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Dari praktikum modul kelima, Implementasi Polimorfisme berguna dalam menciptakan perangkat lunak yang dinamis dan tahan terhadap perubahan (scalable) dengan fitur Method Overriding, suatu subclass dapat merancang ulang logika operasional khusus untuk dirinya tanpa perlu membongkar struktur kode pada kelas induk. Dynamic Method Dispatch serta penggunaan polimorfik memungkinkan sistem untuk berinteraksi dengan berbagai tipe object berbeda

4.2 Link Github

https://github.com/Zarnnonly/PrakPBO_A_2511532001

DAFTAR PUSTAKA

[1] Nurfiah. Modul Praktikum PBO - Java: Praktikum 5 Transaksi Dinamis (Polymorphism). [Diakses: 22 Oktober 2026].